

查找与有序索引

维护多少秩序，才能换取多少查询能力

408 · 数据结构 Topic DS-09

母问题：查询加速从哪里来，又由谁付费？

有序索引的生成链是

Order/Index → Search Reduction → Update Repair → Height Invariant → Cost Tradeoff.

顺序查找几乎不预处理；折半查找要求随机访问和有序区间；BST 把次序放进树路径；平衡树用旋转或分裂把高度约束在对数级。查询越强，通常越要在插入、删除、空间或维护时间上付费。

本册定位与 Owner 边界

- **Owns**：顺序/分块/折半查找、BST、AVL/红黑树的有序性与高度维护、B 树的查找与插删、B+ 树的查找结构。
- **Uses**：Atlas Foundation 成本向量；DS-B02 负责按 workload 比较有序索引与 Hash。
- **Stops**：Hash 映射归 DS10；磁盘块 I/O 的外部排序归 DS12；具体工程库实现只作 Extension。

目录

1	查找契约先固定	3
1.1	顺序、分块与折半：三种秩序预算	3
1.2	ASL 与判定树：把“快”还原成比较次数	3
1.3	顺序查找的三种边界	3
1.4	折半查找：区间不变量与左右边界	3
1.5	答案二分：搜索的是单调可行域	4
1.6	分块查找的两级不变量	4
2	BST：次序下沉到路径	4
2.1	BST 上的三类派生查询：秩、累加与全局边界	5
3	AVL 与红黑树：都控制高度，但修复契约不同	5
4	B 树：用节点扇出降低外存访问高度	6
5	B+ 树：内部节点导航，叶层拥有完整检索序列	6
6	Trie：把前缀共享下沉到路径	7
7	代码与测试证据	7
8	复杂度与概念边界	10

9 做题控制协议

10

1 查找契约先固定

“查找”要先回答返回的是存在性、位置、前驱、首个重复 key 还是范围边界。折半查找依赖一个明确的区间不变量，例如目标若存在必在 $[low, high]$ 中；每轮比较后必须缩小小区间，空区间才是失败证明。重复 key 若不固定取左界/右界，算法结果虽都可能“找到”却不满足接口。

1.1 顺序、分块与折半：三种秩序预算

顺序查找不要求数据有序，逐项排除候选。分块查找把表分成若干块，维护“块间有序、块内可无序”的两级结构：先在索引表定位可能的块，再在块内顺序查找。其平均成本可写成

$$ASL \approx ASL_{index} + ASL_{block},$$

因此块数增加会让索引阶段变长、块内阶段变短；不能只优化其中一项。折半查找则要求整体有序且支持随机访问，每轮用中点把候选区间近似减半。

三者逐步增加维护秩序以缩小查询候选：无索引、稀疏块索引、全局有序随机访问。若数据频繁插入，维持整体连续有序的移动成本可能抵消查询收益。

1.2 ASL 与判定树：把“快”还原成比较次数

查找表是逻辑集合，不等于某一种存储结构；静态表可预先排序和建索引，动态表则要把更新维护写入成本。若成功查找记录 i 的关键字比较次数为 C_i 、访问概率为 P_i ，平均查找长度为

$$ASL_{succ} = \sum_i P_i C_i.$$

等概率时就是比较次数的算术平均。失败查找的平均不能机械除以 n ：基于比较的有序查找由 n 个关键字切出 $n+1$ 个失败区间，判定树有 n 个内部节点和 $n+1$ 个外部节点；应按实际失败入口数加权。散列查找则由哈希函数值域决定失败入口，不能把数组容量直接当分母。

判定树把每次比较画成内部节点，把空区间或空指针画成失败外部节点。成功比较次数是根到内部节点的层数，失败比较次数是到外部节点路径上的内部节点数。这个模型同时解释了顺序查找的单支链、折半查找的近似平衡树，也能验证“复杂度只数关键字比较，不含地址计算”的题目口径。

1.3 顺序查找的三种边界

无序表逐项扫描；逆向扫描可把下标 0 留作失败返回位置。监视哨把目标临时写在哨兵位置，省掉每轮越界判断，但关键字比较次数的渐近量级不变，失败时还会多比较一次。若表有序，扫描到第一个大于目标的 key 即可失败；此时成功 $ASL = (n+1)/2$ ，失败区间的比较次数为 $1, 2, \dots, n, n$ ，而不是把最后一个区间误算成 $n+1$ 。

1.4 折半查找：区间不变量与左右边界

标准模板维护闭区间 $[low, high]$ ：循环条件是 $low \leq high$ ，中点可写为 $low + (high - low)/2$ ，排除中点后必须使用 $mid - 1$ 或 $mid + 1$ ，否则区间可能不缩小。它同时要求全局有序和 $O(1)$ 随机访问；链表即使有序，找中点也要线性走指针。

若重复 key 的接口要求“第一个位置”，命中后不能立即返回，而应记录答案并令 $high = mid - 1$ ；要求“最后一个位置”则令 $low = mid + 1$ 。更一般地，先找第一个满足 $a[i] \geq key$ 的位置 (lower bound)，或第一个满足 $a[i] > key$ 的位置 (upper bound)，再用边界区间判断是否真的命中。任何二分题都应先写谓词的单调性，再决定收缩方向。

折半查找的判定树根到内部节点的层数就是成功比较次数；失败查找落到 $n+1$ 个外部叶，路径上的内部节点数就是失败比较次数。完全二叉判定树高度约为 $\lceil \log_2(n+1) \rceil$ ，但 n 不满一整层时各外部路径长度不同，失败 ASL 不能只写一个高度。给定具体表，直接把根、中点、左右子区间画成树再求和，能避免把“成功 ASL”和“失败 ASL”共用分母。

‘ $\text{mid}=(\text{low}+\text{high})/2$ ’与‘ $\text{mid}=(\text{low}+\text{high}+1)/2$ ’分别偏向下界或上界；当循环不变式是“寻找最后一个满足谓词的位置”时，必须选择与更新式匹配的中点，否则两元素区间可能在‘ $\text{low}=\text{mid}$ ’时不再缩小。代码层面可写成‘ $\text{low} + (\text{high}-\text{low})/2$ ’防止整数加法溢出。

1.5 答案二分：搜索的是单调可行域

有些题不直接查数组位置，而是查最小容量、最短时间或最大允许阈值。先构造判定函数 $P(x)$ ，证明它在候选答案区间上单调，例如“容量 x 能否在限定天数内完成运输”：若 x 可行，则更大容量也可行。此时目标是找第一个可行值，二分模板与 lower bound 相同；若目标是最后一个可行值，则使用相反边界和上取整中点。

完整复杂度为

$$O(C_{\text{check}} \log(U - L + 1)),$$

其中 $[L, U]$ 必须覆盖答案， C_{check} 是一次可行性检查成本。上下界不是随便取：装载容量的下界至少是最大单件重量、上界可取总重量；若 P 并不单调，二分即使代码终止也没有正确性。整数乘加可能溢出时，判定函数应采用扩展类型或提前比较，不能只修正中点公式。

1.6 分块查找的两级不变量

长度为 n 的表分成 b 个块，块内可无序但块间必须整体有序；索引项通常保存该块最大关键字和起始地址，先找第一个 $\max_i \geq \text{key}$ 的块，再在该块内顺序扫描。块内无序换来了插入删除不必维持全表移动，索引查找可用顺序或折半。若每块 s 个元素且索引、块内都用顺序查找，成功查找平均成本近似

$$\text{ASL} = \frac{b+1}{2} + \frac{s+1}{2}, \quad bs = n.$$

由均值不等式可知两段工作量平衡时最优，即 $b = s = \sqrt{n}$ ；索引改为折半后，最优块长会改变，不能沿用这个结论。

2 BST：次序下沉到路径

BST 对每个节点保持左子树 key 小于节点、右子树 key 大于节点（重复值策略需显式约定）。查找、插入、删除都沿一条根叶路径；复杂度是 $O(h)$ ，不是无条件 $O(\log n)$ 。退化链的高度为 n ，所以平衡条件不是装饰而是复杂度证明的前提。

删除的三种结构分支

叶节点直接删除；单孩子用孩子替代节点；双孩子从中序后继/前驱取替代 key，再在其原位置删除。每一步都要恢复局部 BST 有序性，不能只交换值而忘记释放或接回子树。

BST 的中序遍历必须得到非降 key 序列，这是验证顺序不变量的最短证据；若重复 key 采用“相等统一放右侧”等规则，验证时也必须使用同一严格/非严格关系。求最近邻、前驱或后继不必遍历整棵树：沿搜索路径维护当前最佳候选即可。构造树时，先序/后序给根，中序把左右子树切开；若 key 重复，序列通常不能唯一确定一棵树，题面必须给出额外规则。

2.1 BST 上的三类派生查询：秩、累加与全局边界

BST 的有序性不只用于查找 key，还能把一次中序/反中序扫描变成派生状态：

- **第 k 小**：中序访问天然按非降序产生 key，维护计数器，计数到 k 即可停止；若树高为 h ，沿路径并访问前 k 个节点的成本可写为 $O(h+k)$ ，不提前停止则为 $O(n)$ 。重复 key 的“第 k 个”必须先约定按节点还是按不同 key 计数。
- **大于等于当前值的累加树**：反中序（右—根—左）按降序访问，维护累计和并把当前 key 替换为“原值加上已见更大值”；必须先保存原 key，不能在比较结构仍需使用时覆盖后再递归。
- **验证是否为 BST**：不能只检查每个节点与直接孩子的局部大小；应沿递归传递严格的 $(lower, upper)$ 开区间，或检查中序序列是否满足题目约定的严格/非严格单调性。局部孩子比较无法发现“孙节点越过祖先边界”的反例。

这些应用的共同前提是树结构确实满足 BST 不变量。若输入只是任意二叉树，先做验证或改用显式排序；不能因为“写了中序”就自动拥有有序语义。

3 AVL 与红黑树：都控制高度，但修复契约不同

AVL 约束任一节点左右子树高度差不超过 1。插入后从修改点向根寻找第一个失衡祖先，根据较高子树方向形成 LL/RR/LR/RL；旋转后必须同时恢复 BST 中序有序和高度约束。

四种插入失衡可压缩为两次选择：先看失衡节点的重子树方向，再看重子树根的重子树方向。LL 与 RR 各一次单旋，LR 与 RL 先对孩子反向旋转再对失衡节点旋转。删除可能让多层祖先连续失衡，不能套用“插入只修第一个节点就停止”的经验；每次旋转后都要重新计算高度。

AVL 高度界可用最小节点数递推验证。令 $N(h)$ 为高度为 h 的 AVL 树最少节点数，则

$$N(h) = 1 + N(h-1) + N(h-2), \quad N(0) = 1, N(1) = 2,$$

因此 $N(h)$ 至少按 Fibonacci 数增长，反推得到 $h = O(\log n)$ 。这个递推说明“平衡因子不超过 1”确实足以给出对数高度；它不等于每棵 AVL 都是完全树。

红黑树把严格高度平衡放宽为颜色约束：根与空叶为黑；红节点的孩子不能为红；从任一节点到其后代空叶的每条路径具有相同黑高。由此最长根叶路径不超过最短路径两倍，树高仍为 $O(\log n)$ 。插入冲突按叔节点颜色和结构方向决定变色或旋转；删除若移走黑色会产生黑高亏损，修复目标是把亏损上移、吸收或消除。

红黑树的五条颜色不变量应和“空叶也算黑”一起检查：根黑、外部 NIL 黑、红节点不得有红孩子、任一节点到所有后代 NIL 的黑节点数相同、仍保持 BST 有序。它的旋转比 AVL 少严格度，但通常更新路径更短；不能由“高度是对数级”反推出两者的旋转次数或颜色状态相同。

红黑树插入时，新节点先染红以保持黑高不变；若父节点为黑则结束，若父为红则看叔节点：叔红时父叔变黑、祖父变红并把冲突上移；叔黑时按 LL/LR/RR/RL 方向旋转并重新着色。删除时真正困难的是删除黑节点造成的“额外黑色”：兄弟为红先旋转把情形转成兄弟为黑；兄弟黑且两个孩子黑则把黑色上移；若远侄为黑、近侄为红先对兄弟旋转；远侄为红时借用其颜色完成最终旋转。纸面题应始终检查五条不变量，而不是只背某个 case 的图。

AVL 与红黑树不能共享旋转口诀

AVL 的触发量是高度差，红黑树的触发量是颜色与黑高。二者旋转都保持 BST 次序，但合法状态不同；只看到“失衡就旋转”无法判断变色与停止条件。

4 B 树：用节点扇出降低外存访问高度

设 m 阶 B 树，每个节点至多有 m 个孩子、至多 $m - 1$ 个关键字；除根外的非叶节点至少有 $\lceil m/2 \rceil$ 个孩子，所有叶节点位于同一层。节点内关键字有序并划分孩子区间。查找先在节点内确定区间，再沿一个孩子下降；昂贵的是访问多少个节点/块。

“阶”的教材口径可能用最大孩子数 m ，也可能用最大关键字数；下限、分裂位置和高度计算必须先抄出题目定义。若采用最大孩子数口径，非根节点关键字下限为 $\lceil m/2 \rceil - 1$ ；根可少于此数但不能违反空树/单孩子的特殊条件。节点内查找既可顺序比较也可折半，外存复杂度通常只计块访问，不能把节点内比较次数与 I/O 次数混为一项。

插入总在叶层落位。节点溢出时把中间关键字上升父节点，左右关键字分成两个合法节点；父节点若再溢出就继续向上，根分裂会使树高增加 1。删除后若节点低于下限，先向兄弟借一个关键字并经父节点重分配；不能借才与兄弟及父分隔关键字合并。若根最终无关键字且只有一个孩子，用该孩子替代根，树高减 1。

统一不变量是：节点占用合法、节点内有序、孩子区间正确、所有叶同层。分裂和合并是在保持这些不变量下控制高度。

5 B+ 树：内部节点导航，叶层拥有完整检索序列

B+ 树把记录或记录指针集中在叶节点；非叶节点只保存导航分隔 key，关键字可能在内部和叶层重复。所有叶节点按 key 顺序链接，因此等值查找沿根到叶完成，范围查找定位起点后沿叶链扫描。

在本题常用的“最大关键字复写”口径中，B+ 树内部节点的关键字数等于子树数，内部索引 key 只是分界副本；即使在内部节点比较相等，也必须继续下降到叶节点取真正记录。叶节点分裂时把分界 key 复制到父节点而不从叶中删除；叶链因此始终保有完整有序序列。删除叶中最大 key 时，父节点副本可继续作为合法分界，不必把它误当成记录。

B 树和 B+ 树查找都为 $O(\log_m n)$ ，B+ 树的优势主要是更大扇出、固定根到叶路径、 $O(\log_m n + k)$ 的范围扫描和沿叶链的全表遍历，而不是渐近阶发生改变。磁盘页预算应把“关键字、指针、记录”分别计数：内部节点不放记录，才能在同一页容纳更多导航项。

概念 A	≠ 概念 B	真正判据与题面信号	混淆后果
B 树	≠ B+ 树	B 树各层可命中记录；B+ 树内部只导航、叶链承载全体 key	误判成功层次和范围扫描路径
树的阶	≠ 节点关键字数	阶限制最大孩子数，关键字上限通常少 1	占用上下限算错
分裂	≠ 旋转借位	分裂制造新节点；借位在相邻节点和父分隔 key 间重分配	高度变化判断错

6 Trie：把前缀共享下沉到路径

Trie 把字符串的第 i 个字符放在根到深度 $i + 1$ 的边上；节点至少包含孩子映射和终止标记 `isEnd`。插入、完整词查找与前缀查找都沿字符路径前进，时间为 $O(L)$ ，其中 L 是查询串长度，而不是词典中的词数。`isEnd` 不能省略：存在路径 `app`→`apple` 不代表短串本身已被插入。

孩子表示决定空间与常数。固定小字符集可用定长数组，转移接近直接定位但每个节点都支付字母表空间；稀疏或 Unicode 字符集可用 Hash/有序映射，只为真实边付费但增加映射成本。总节点数不超过所有已插入字符串长度之和加根节点，前缀共享越多越节省。删除一个词时先清除终止标记，只有节点既非其他词终点又没有孩子时才能向上回收，不能删除共享前缀。

Trie Owns 前缀路径语义，不替代 DS10 的精确等值 Hash，也不替代 B+ 树的有序范围索引。若要自动补全，可定位前缀节点后遍历其子树；若要统计前缀频次，可在路径节点维护经过计数，但插入、删除必须同步维护该派生状态。

7 代码与测试证据

完整实现位于 `code/ds09_ordered_index.hpp`，测试位于 `code/ds09_ordered_index_test.cpp`。本册用边界明确的 BST 作为可执行核心，平衡树和 B/B+ 的分裂规则保留为机制扩展。

```

1
2 class BinarySearchTree {
3 public:
4     bool contains(int key) const { return contains(root_.get(), key); }
5     void insert(int key) { insert(root_, key); }
6     bool erase(int key) { return erase(root_, key); }
7     std::vector<int> inorder() const { std::vector<int> result; inorder(root_.get(), result); return result; }
8
9 private:
10    std::unique_ptr<Node> root_;
11    static bool contains(const Node* node, int key) { if (!node) return false; if
        (key == node->key) return true; return key < node->key ? contains(node->left
        .get(), key) : contains(node->right.get(), key); }
12    static void insert(std::unique_ptr<Node>& node, int key) { if (!node) { node
        = std::make_unique<Node>(Node{key, nullptr, nullptr}); return; } if (key <
        node->key) insert(node->left, key); else if (key > node->key) insert(node->
        right, key); }
13    static bool erase(std::unique_ptr<Node>& node, int key) {
14        if (!node) return false;
15        if (key < node->key) return erase(node->left, key);
16        if (key > node->key) return erase(node->right, key);
17        if (!node->left) { node = std::move(node->right); return true; }
18        if (!node->right) { node = std::move(node->left); return true; }
19        Node* successor = node->right.get(); while (successor->left) successor =
        successor->left.get();
20        node->key = successor->key; return erase(node->right, successor->key);
21    }
22    static void inorder(const Node* node, std::vector<int>& result) { if (!node)
        return; inorder(node->left.get(), result); result.push_back(node->key);

```

```

        inorder(node->right.get(), result); }
23 };
24
25 class AvlTree {
26 public:
27     bool contains(int key) const { return contains(root_.get(), key); }
28     void insert(int key) { root_ = insert(std::move(root_), key); }
29     std::size_t height() const { return height(root_.get()); }
30     std::vector<int> inorder() const { std::vector<int> result; inorder(root_.get(), result); return result; }
31     bool invariant() const { int actual = 0; return check(root_.get(), nullptr, nullptr, actual); }
32
33 private:
34     struct AvlNode { int key; int height = 1; std::unique_ptr<AvlNode> left,
        right; };
35     std::unique_ptr<AvlNode> root_;
36     static int height(const AvlNode* node) { return node == nullptr ? 0 : node->height; }
37     static void refresh(AvlNode* node) { node->height = 1 + std::max(height(node->left.get()), height(node->right.get())); }
38     static int balance(const AvlNode* node) { return node == nullptr ? 0 : height(node->left.get()) - height(node->right.get()); }
39     static std::unique_ptr<AvlNode> rotate_right(std::unique_ptr<AvlNode> node) {
        auto pivot = std::move(node->left); node->left = std::move(pivot->right);
        refresh(node.get()); pivot->right = std::move(node); refresh(pivot.get());
        return pivot; }
40     static std::unique_ptr<AvlNode> rotate_left(std::unique_ptr<AvlNode> node) {
        auto pivot = std::move(node->right); node->right = std::move(pivot->left);
        refresh(node.get()); pivot->left = std::move(node); refresh(pivot.get());
        return pivot; }
41     static std::unique_ptr<AvlNode> insert(std::unique_ptr<AvlNode> node, int key) {
42         if (!node) return std::make_unique<AvlNode>(AvlNode{key, 1, nullptr, nullptr});
43         if (key < node->key) node->left = insert(std::move(node->left), key);
44         else if (key > node->key) node->right = insert(std::move(node->right), key);
45         else return node;
46         refresh(node.get()); const int factor = balance(node.get());
47         if (factor > 1 && key < node->left->key) return rotate_right(std::move(node));
48         if (factor < -1 && key > node->right->key) return rotate_left(std::move(node));
49         if (factor > 1 && key > node->left->key) { node->left = rotate_left(std::move(node->left)); return rotate_right(std::move(node)); }
50         if (factor < -1 && key < node->right->key) { node->right = rotate_right(std::move(node->right)); return rotate_left(std::move(node)); }
51         return node;
52     }
53     static bool contains(const AvlNode* node, int key) { if (!node) return false;
        if (key == node->key) return true; return key < node->key ? contains(node->

```



```

    left.get(), key) : contains(node->right.get(), key); }
54 static void inorder(const AvlNode* node, std::vector<int>& result) { if (!
    node) return; inorder(node->left.get(), result); result.push_back(node->key
    ); inorder(node->right.get(), result); }
55 static bool check(const AvlNode* node, const int* lower, const int* upper, int
    & actual_height) {
56     if (!node) { actual_height = 0; return true; }
57     if ((lower && node->key <= *lower) || (upper && node->key >= *upper))
    return false;
58     int left_height = 0, right_height = 0;
59     if (!check(node->left.get(), lower, &node->key, left_height) || !check(
    node->right.get(), &node->key, upper, right_height)) return false;
60     actual_height = 1 + std::max(left_height, right_height);
61     return node->height == actual_height && std::abs(left_height - right_height
    ) <= 1;
62 }
63 };
64 } // namespace ipara::ds09
65
66 #endif

```

代码清单 1: BST 查找、插入、删除与有序输出

```

1  assert(!tree.contains(4));
2  tree.insert(5); tree.insert(3); tree.insert(7); tree.insert(6); tree.insert
    (8); tree.insert(3);
3  assert(tree.contains(6));
4  assert((tree.inorder() == std::vector<int>{3, 5, 6, 7, 8}));
5  assert(tree.erase(3)); assert(tree.erase(5)); assert(!tree.erase(42));
6  assert((tree.inorder() == std::vector<int>{6, 7, 8}));
7  ipara::ds09::AvlTree avl;
8  for (int key : {30, 20, 10, 25, 28, 40, 50}) { avl.insert(key); assert(avl.
    invariant()); }
9  assert(avl.contains(28) && !avl.contains(99));
10 assert((avl.inorder() == std::vector<int>{10, 20, 25, 28, 30, 40, 50}));
11 assert(avl.height() <= 3);
12 }

```

代码清单 2: 空树、重复 key 与删除分支测试

```

1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic code/
    ds09_ordered_index_test.cpp -o /tmp/ds09_test
2 /tmp/ds09_test

```

8 复杂度与概念边界

结构/操作	平均或保证	最坏	依赖的不变量	维护代价
顺序查找	$O(n)$	$O(n)$	无序也可	几乎无预处理
折半查找	$O(\log n)$	$O(\log n)$	有序 + 随机访问	插入移动
BST	$O(h)$	$O(n)$	局部有序	可能退化
平衡树	$O(\log n)$	$O(\log n)$	高度/颜色约束	旋转/分裂

9 做题控制协议

先写返回契约和区间/树不变量，再看输入是否有序、是否支持随机访问、更新是否频繁。看到 Hash 适合精确等值但不适合范围查询；看到动态有序更新，才考虑平衡索引。每个复杂度都把高度或有序前提写出来。

压缩复原

五问：查询要返回什么？候选区间怎样缩小？秩序存在哪里？更新破坏哪个不变量？维护成本由谁承担？